

Documentación Pepmultitools

Preparado por Sofia Andrade Palacio

Correo: soandradep@unal.edu.co

Celular: 3165357318

▼ Descripción General

PepMultiTools es una herramienta computacional diseñada para la búsqueda, clasificación y generación de péptidos basados en sus propiedades fisicoquímicas.

Su objetivo principal es facilitar la búsqueda de péptidos a partir de proteínas considerando sus propiedades fisicoquímicas, predecir la probabilidad de actividad antimicrobiana en péptidos y generar secuencias de péptidos antimicrobianos sintéticos.

▼ Características Principales

La herramienta está compuesta por tres módulos:

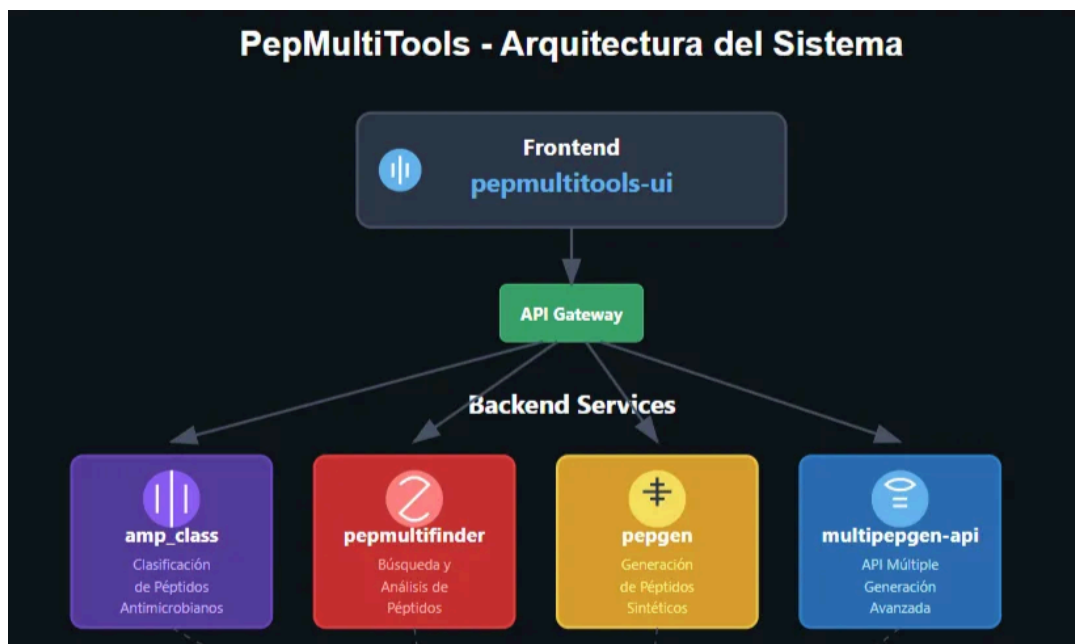
- **PepMultiFinder 2.0 (Búsqueda de Péptidos en Proteomas o Listas de Proteínas):** Permite la búsqueda de péptidos candidatos a ser antimicrobianos a partir de proteomas proporcionados en formato FASTA. La búsqueda se realiza explorando subsecuencias de aminoácidos dentro de un rango de longitudes definido por el usuario, con la posibilidad de aplicar filtros basados en propiedades fisicoquímicas también personalizadas.
- **AmpClass 1.0 (Clasificación de Péptidos):** En este módulo, el usuario proporciona un conjunto de secuencias de péptidos de interés y recibe una predicción cuantitativa de la probabilidad de que cada secuencia tenga actividad antimicrobiana, basada en modelos de aprendizaje supervisado preentrenados.
- **PepGen 1.0 (Generación de Péptidos):** Permite la generación de péptidos sintéticos con alta probabilidad de actividad antimicrobiana. Los usuarios

pueden definir parámetros como la cantidad de péptidos a generar y su rango de longitud.

- **Multipepgen:** Es una red neuronal Conditional GAN con celdas LSTM para la generación de secuencias sintéticas de péptidos antimicrobianos.

▼ Arquitectura

PepMultiTools está diseñado bajo una arquitectura de microservicios. La aplicación cuenta con un servicio principal, que es la interfaz de usuario desarrollada en **React**. Desde esta interfaz, se realizan llamados a cuatro microservicios independientes. Tres de ellos desarrollados con Flask(amp_class, pepmultifinder y pepgen) y uno con FastAPI (multipepgen).



▼ Tecnologías Utilizadas

- **Backend:** Flask y FastAPI
- **Frontend:** React y Dash
- Redis
- Docker

▼ Configuración para desarrollo

▼ Requisitos Previos

- Python 3.9+
- Docker y Docker Compose
- Redis (para la gestión de tareas en background)

▼ Pasos de Instalación

1. Interfaz de usuario

- Clonar el repositorio

```
git clone https://github.com/Pepmultitools-UNAL/pepmultitools-ui.git
cd pepmultitools-ui
```

- docker-compose.yml

```
version: '3'
services:
  app:
    build: .
    ports:
      - "8000:80" # Servimos en el puerto 8000
    environment:
      - NODE_ENV=production

    networks:
      - peptools_network

networks:
  peptools_network:
    external: true
```

```
docker-compose build
docker network create peptools_network
```

```
docker-compose up
```

2. Servicios backend Pepmultifinder, Pepgen y Ampclass

- Clonar el repositorio:

```
git clone https://github.com/Pepmultitools-UNAL/https://github.com/Pepmultitools-UNAL/pepmultifinder.git
cd pepmultifinder
```

- Configuración de variables de entorno: Crea un archivo .env con las variables necesarias

```
FLASK_APP=wsgi.py
FLASK_DEBUG=True
SECRET_KEY=randomstringofcharacters
LESS_BIN=/usr/local/bin/lessc
ASSETS_DEBUG=False
LESS_RUN_IN_DEBUG=False
COMPRESSOR_DEBUG=True
FLASK_RUN_PORT=5001
SERVER_NAME=localhost:5001
DEBUG=True
REDIS_URL=redis://redis:6379/0
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USE_SSL=False
MAIL_USERNAME=peptools_med@unal.edu.co
MAIL_PASSWORD=nwis rsdj orco lnex
```

- Configuración de docker-compose

```
version: '3'
services:
  pepmultifinder:
    build: .
    image: master-image
    env_file:
      - .env
    ports:
      - "5000:5000"
    volumes:
      - ./app
    depends_on:
      - redis
    networks:
      - peptools_network

  pepmultifinder_worker:
    image: pepmultifinder-image
    volumes:
      - ./app
    depends_on:
      - redis
    command: flask rq worker
    networks:
      - peptools_network
    links:
      - redis
  redis:
    image: redis
    ports:
      - "6379:6379"
    networks:
      - peptools_network
```

```
networks:
  peptools_network:
    external: true
```

- Construcción y ejecutar con docker

```
docker-compose build
docker network create peptools_network
docker-compose up
```

Se siguen exactamente los mismos pasos para instalar los dos servicios faltantes, Ampclass y Pepgen, solo cambias los links de los repositorios:

```
git clone https://github.com/Pepmultitools-UNAL/pepgen.git
git clone https://github.com/Pepmultitools-UNAL/amp_class.git
```

1. Servicio backend Multipepgen-api

- Clonar el repositorio:

```
git clone https://github.com/Anorpe/multipepgen-api.git
cd multipepgen-api
```

#Build

```
docker build -t multipepgen-api-image .
```

Run

```
docker run --name multipepgen-api-container -p 9000:9000 multipepgen-api-image
```

Ejecutar sin bloquear la terminar

```
docker run --name multipepgen-api-container -p 9000:9000 -d multipepgen-api-image
```

▼ Uso de las APIs

▼ Pepmultifinder

- **Método:** `POST`
- **URL:** `/search`
- Parámetros para la solicitud:
 - **Tipo:** `multipart/form-data`
 - **Cuerpo (`FormData`):**

Nombre	Tipo	Descripción
<code>email</code>	<code>string</code>	Correo electrónico del usuario donde se enviarán los resultados.
<code>fasta_file</code>	<code>file</code>	Archivo en formato FASTA con las secuencias proteicas a analizar.
<code>min_longitud</code>	<code>number</code>	Longitud mínima de las secuencias a extraer.
<code>max_longitud</code>	<code>number</code>	Longitud máxima de las secuencias a extraer.
<code>min_isoelectrico</code>	<code>number</code>	Valor mínimo del punto isoeléctrico.
<code>max_isoelectrico</code>	<code>number</code>	Valor máximo del punto isoeléctrico.
<code>min_polar_angles</code>	<code>number</code>	Ángulo polar mínimo.
<code>max_polar_angles</code>	<code>number</code>	Ángulo polar máximo.
<code>min_boman</code>	<code>number</code>	Índice de Boman mínimo.
<code>max_boman</code>	<code>number</code>	Índice de Boman máximo.
<code>min_momento</code>	<code>number</code>	Momento dipolar mínimo.
<code>max_momento</code>	<code>number</code>	Momento dipolar máximo.
<code>min_wimley</code>	<code>number</code>	Índice de Wimley-White mínimo.
<code>max_wimley</code>	<code>number</code>	Índice de Wimley-White máximo.
<code>min_carga</code>	<code>number</code>	Carga eléctrica mínima de las secuencias.

max_carga	number	Carga eléctrica máxima de las secuencias.
min_hidrofobico	number	Hidrofobicidad mínima.
max_hidrofobico	number	Hidrofobicidad máxima.
escala	string	Escala utilizada para calcular propiedades fisicoquímicas.
apply_helices	boolean	Indica si se aplicarán filtros de estructuras helicoidales (<code>true</code> o <code>false</code>).
metodo_helices	string	Método a utilizar para la detección de hélices.
exclude_aminoacidos	string	Lista de aminoácidos a excluir del análisis.

▼ Pepgen

- **Método:** `POST`
- **URL:** `/generator`
- **Parámetros de la solicitud:**
 - **Tipo:** `multipart/form-data`
 - **Cuerpo (`FormData`):**

Nombre	Tipo	Descripción
email	string	Correo electrónico del usuario donde se enviarán los resultados.
number_secuencias	number	Número de péptidos a generar.
min_longitud	number	Longitud mínima de los péptidos generados.
max_longitud	number	Longitud máxima de los péptidos generados.

▼ Ampclass

- **Método:** `POST`
- **URL:** `/predictor`
- **Parámetros de la solicitud:**
 - **Tipo:** `multipart/form-data`
 - **Cuerpo (`FormData`):**

Nombre	Tipo	Descripción
<code>email</code>	<code>string</code>	Correo electrónico del usuario donde se enviarán los resultados.
<code>fasta_file</code>	<code>File</code>	Archivo .fasta
<code>include_secuencias</code>	<code>string</code>	Secuencias separadas por (,)
<code>umbral</code>	<code>number</code>	Punto que se usa para decidir la clasificación.

▼ Multipepgen

- **Método:** `POST`
- **URL:** `/generate`
- **Parámetros de la solicitud:**
 - **Tipo:** `application/json`
 - **Cuerpo (`JSON`):**

Nombre	Tipo	Descripción
<code>n_gens</code>	<code>number</code>	Número de péptidos que se desean generar.
<code>functionalities</code>	<code>string[]</code>	Lista de funcionalidades que deben tener los péptidos generados. Ejemplo: ["antimicrobiano", "antiviral"].
<code>filters</code>	<code>object</code>	Filtros adicionales para restringir las características de los péptidos generados.

Detalles de `filters` :

Nombre	Tipo	Descripción
<code>excludedAminoacids</code>	<code>string</code>	Aminoácidos que se deben excluir de los péptidos generados. Se pasan como una cadena separada por comas (por ejemplo, <code>"A,C,D"</code>).
<code>sizeRange</code>	<code>[number, number]</code>	Rango de tamaños (longitud) de los péptidos a generar. Ejemplo: <code>[5, 10]</code> genera péptidos de entre 5 y 10 aminoácidos.
<code>probability</code>	<code>number</code>	Probabilidad mínima de que el péptido generado cumpla con las características deseadas. El valor debe estar entre 0 y 1

(por ejemplo, `0.8` indica que el péptido debe tener al menos un 80% de probabilidad de cumplir con los criterios).

▼ Estructura de carpetas

▼ Estructura del Frontend

pepmultitools-ui tiene la siguiente estructura:

- 📁 `src/` - Carpeta principal del código fuente.
 - ├── 📁 `assets/` - Contiene archivos estáticos como imágenes.
 - ├── 📁 `components/` - Contiene componentes reutilizables.
 - ├── 📁 `forms/` - Formularios para los distintos servicios.
 - ├── `AmpclassForm.tsx` - Formulario para AmpClass.
 - ├── `MultifinderForm.tsx` - Formulario para PepMultiFinder.
 - ├── `PepgenForm.tsx` - Formulario para PepGen.
 - ├── 📁 `multipepgen/` -
 - ├── `CreditModal.tsx` - Modal de créditos.
 - ├── `DescriptionModal.tsx` - Modal de descripción.
 - ├── `Footer.tsx` - Pie de página.
 - ├── 📁 `hooks/` - Contiene hooks personalizados.
 - ├── `useAmpclassForm.ts` - Hook para gestionar el formulario de AmpClass.
 - ├── `useMultifinderForm.ts` - Hook para gestionar el formulario de PepMultiFinder.
 - ├── `usePepgenForm.ts` - Hook para gestionar el formulario de PepGen.
 - ├── `useMultipepgen.ts` -
 - ├── 📁 `pages/` - Contiene las páginas principales del frontend.
 - ├── `Ampclass.tsx` - Página para la funcionalidad de AmpClass.
 - ├── `Home.tsx` - Página principal de la aplicación.
 - ├── `Pepgen.tsx` - Página de generación de péptidos (PepGen).
 - ├── `Pepmultifinder.tsx` -

- | └─ `Multipepgen.tsx` -
- | └─ `PepgenDownload.tsx` - Página de descarga de resultados de PepGen.
- | └─ `PepgenResults.tsx` - Página de resultados de PepGen.
- | └─ `PepmultifinderResults.tsx` - Página de resultados de PepMultiFinder.
- └─  `services/` - Contiene los servicios que manejan las solicitudes al backend.
- | └─ `ampclassService.ts` - Servicio para interactuar con AmpClass.
- | └─ `pepgenService.ts` - Servicio para interactuar con PepGen.
- | └─ `pepmultifinderService.ts` - Servicio para interactuar con PepMultiFinder.
- | └─ `multipepgenService.ts` - Servicio para interactuar con Multipepgen.
- └─  `styles/` - Contiene los archivos CSS.
- | └─  `components/` - Estilos específicos de componentes.
- | └─ `App.css` - Estilos globales de la aplicación.
- └─ `App.tsx` - Archivo principal de la aplicación React.

Este frontend está organizado de forma modular:

- **Componentes reutilizables** (`components/`) para formularios y modales.
- **Hooks personalizados** (`hooks/`) que manejan la lógica de los formularios.
- **Páginas principales** (`pages/`) que renderizan las vistas según la funcionalidad.
- **Servicios** (`services/`) que gestionan las llamadas a los endpoints del backend.
- **Estilos CSS** (`styles/`) para la personalización de la UI.

▼ Estructura del Backend

Pepmultifinder, Pepgen y Ampclass tienen la siguiente estructura:

-  `src/` - Carpeta principal del código fuente.
- └─  `application/` - Contiene la lógica de aplicación.

- | —  `config/` - Archivos de configuración global de la aplicación.
- | —  `domain/` - Define las entidades del dominio del negocio.
- | —  `infrastructure/` - Contiene la infraestructura del proyecto.
- | | —  `api/` - Define los endpoints de la API.
- | | —  `config/` - Configuraciones específicas del backend.
- | | —  `persistence/` - Módulos para gestionar la persistencia de datos.
- | | —  `plotlydash/` - Integración con Plotly Dash para visualizaciones.
- | | —  `ui/` - Contiene elementos relacionados con la interfaz de usuario (si los hay).
- | | —  `utils/` - Funciones y utilidades auxiliares.
- | | — `__init__.py` - Permite que la carpeta sea tratada como un módulo de Python.
- | | — `app.db` - Base de datos de la aplicación.
- | | — `wsgi.py` - **Punto de entrada de la aplicación** para el servidor WSGI.
- | | — `.env` - Archivo de variables de entorno.

Este backend sigue una arquitectura limpia y modular:

- **Capa de aplicación** (`application/`) con la lógica de negocio.
- **Capa de dominio** (`domain/`) que representa las entidades clave.
- **Capa de infraestructura** (`infrastructure/`):
 - `api/` para la API REST.
 - `persistence/` para la base de datos.
 - `plotlydash/` para gráficos y dashboards.
 - `ui/` (posiblemente para vistas o templates).
 - `utils/` con funciones auxiliares.
- **Archivo** `wsgi.py` que es el punto de entrada de la aplicación.

multipepgen-api

models/	#  Directorio de modelos
.DS_Store	
.gitignore	
.pre-commit-config.yaml	
Dockerfile	# Configuración de contenedor Docker
README.md	# Documentación del proyecto
main.py	# Archivo principal de Python
poetry.lock	# Archivo de bloqueo de dependencias
pyproject.toml	# Configuración del proyecto Python
requirements.txt	# Lista de dependencias (pip)
requirements_docker.txt	# Dependencias específicas para Docker

▼ Configuración para despliegue

▼ Datos del servidor

```
ssh laboratorio@168.176.97.132
Clave: Laboratorio2020*
```

▼ Pasos para acceder al servidor

```
(base) labprocesosbiom@GGRP0T2:~$ ssh laboratorio@168.176.97.132 worker pro
laboratorio@168.176.97.132's password: 13:09:17 [notice] l#1: start worker pro
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 4.15.0-213-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Tue Jun 24 08:10:13 -05 2025

System load:          0.11
Usage of /:            30.1% of 342.49GB
Memory usage:         15%
Swap usage:           0%
Processes:            535
Users logged in:      1
IP address for ens160: 168.176.97.132
IP address for docker0: 172.17.0.1
IP address for br-b9e9514f98c5: 172.18.0.1
IP address for br-fd07b5c46a0b: 172.22.0.1
IP address for br-0049e128a964: 172.29.0.1

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.
  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

El mantenimiento de seguridad expandido para Infrastructure está desactivado.
Se pueden aplicar 0 actualizaciones de forma inmediata.
205 actualizaciones de seguridad adicionales se pueden aplicar con ESM Infra.
Aprenda más sobre cómo activar el servicio ESM Infra for Ubuntu 18.04 at
https://ubuntu.com/18-04

Last login: Wed Jun 18 17:12:19 2025 from 168.176.114.47
```

```
(base) laboratorio@microbiologia:~$ ls
anaconda3  docker-compose.yml  ejecucionContenedores.sh  generacion_datasets  multipeppen-env  pepmultitools  pepmultitools-docker  requirements.txt  traductor
(base) laboratorio@microbiologia:~$ cd pepmultitools
(base) laboratorio@microbiologia:~/pepmultitools$ ls
app_class  docker-compose.redis.yml  multipeppen-api  multipeppen-app  pepgen  pepmultifinder  pepmultitools-ui  ports
(base) laboratorio@microbiologia:~/pepmultitools$
```

Se ingresa el comando `sudo su` para cambiar al usuario root (super usuario) y poder ejecutar comandos necesarios.

Con docker ps veo todos los contenedores corriendo en el servidor.

```
(base) laboratorio@microbiologia:~/pepmultitools$ sudo su
(sudo) contraseña para laboratorio:
root@microbiologia:/home/laboratorio# docker ps --format '{{.ContainerID}}\n{{.Image}}\n{{.Command}}\n{{.Created}}\n{{.Status}}\n{{.Ports}}\n{{.Names}}'
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS        PORTS                               NAMES
378d7c16948    pepmultitools-ui-app               "/docker-entrypoint..."              18 minutes ago Up 18 minutes 0.0.0.0:8000->80/tcp, :::8000->80/tcp pepmultitools-ui-app-1
c0816c33d45    ampclass-image                     "/entrypoint.sh flas..."             8 days ago    Up 6 days    5003/tcp, 6381/tcp                  ampclass-ampclass-1
8f6dc3b821f1    ampclass-image                     "/entrypoint.sh /ata..."             8 days ago    Up 6 days    80/tcp, 5003/tcp, 6381/tcp          ampclass-ampclass-1
21f839a3351    pepgen-image                       "/entrypoint.sh flas..."             2 weeks ago   Up 2 weeks    5002/tcp                            pepgen-peppen-1
8e533747685    pepgen-image                       "/entrypoint.sh /ata..."             2 weeks ago   Up 2 weeks    80/tcp, 5002/tcp                    pepgen-peppen-1
eaa346ec2e44    multipeppen-app-multipeppen-app    "poetry run unicorn..."              2 weeks ago   Up 2 weeks    80/tcp                              multipeppen-app-container
7e7808b3b0ec    multipeppen-api-image              "python main.py"                       2 weeks ago   Up 2 weeks    80/tcp                              multipeppen-api-container
e0b3cec86e7e    pepmultifinder-image               "/entrypoint.sh flas..."             2 weeks ago   Up 2 weeks    80/tcp, 5001/tcp                    pepmultifinder-pepmultifinder-1
e328c0f9e08    pepmultifinder-image               "/docker-entrypoint.s..."             3 weeks ago   Up 3 weeks    0.0.0.0:6379->6379/tcp, :::6379->6379/tcp pepmultifinder-pepmultifinder-1
55688610c22e    invertep-traductor                 "unicorn -bind 0.0.0.0..."           6 weeks ago   Up 6 weeks    80/tcp                              invertep-traductor-1
855ae2ee22cd    jwilder/nginx-proxy                "/app/docker-entryp..."               6 weeks ago   Up 6 weeks    0.0.0.0:80->80/tcp, :::80->80/tcp, 0.0.0.0:443->443/tcp, :::443->443/tcp invertep-nginx-proxy-1
4319f530a52    jwilder/nginx-proxy-companion      "/bin/bash /app/entr..."              6 weeks ago   Up 6 weeks                                     invertep_letsencrypt-1
root@microbiologia:/home/laboratorio#
```

▼ Configuración de variables de entorno

Los archivos .env de los servicios Pepgen, Ampclass y Pepmultifinder deben de contener lo siguiente:

Pepgen

```
FLASK_APP=wsgi.py
FLASK_DEBUG=True
SECRET_KEY=randomstringofcharacters
LESS_BIN=/usr/local/bin/lessc
ASSETS_DEBUG=False
LESS_RUN_IN_DEBUG=False
COMPRESSOR_DEBUG=True
FLASK_RUN_PORT=5002
SERVER_NAME=pepgen.medellin.unal.edu.co
DEBUG=True
REDIS_URL=redis://redis-peptools:6379/0
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USE_SSL=False
MAIL_USERNAME=peptools_med@unal.edu.co
MAIL_PASSWORD=nwis rsdj orco lnex
```

Pepmultifinder

```
FLASK_APP=wsgi.py
FLASK_DEBUG=True
SECRET_KEY=randomstringofcharacters
LESS_BIN=/usr/local/bin/lessc
ASSETS_DEBUG=False
LESS_RUN_IN_DEBUG=False
COMPRESSOR_DEBUG=True
FLASK_RUN_PORT=5001
```

```
SERVER_NAME=pepmultifinder.medellin.unal.edu.co
DEBUG=True
REDIS_URL=redis://redis-peptools:6379/0
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USE_SSL=False
MAIL_USERNAME=peptools_med@unal.edu.co
MAIL_PASSWORD=nwis rsdj orco lnex
```

Ampclass

```
FLASK_APP=wsgi.py
FLASK_DEBUG=True
SECRET_KEY=randomstringofcharacters
LESS_BIN=/usr/local/bin/lessc
ASSETS_DEBUG=False
LESS_RUN_IN_DEBUG=False
COMPRESSOR_DEBUG=True
FLASK_RUN_PORT=5003
SERVER_NAME=ampclass.medellin.unal.edu.co
DEBUG=True
REDIS_URL=redis://redis-peptools:6379/0
MAIL_SERVER=smtp.gmail.com
MAIL_PORT=587
MAIL_USE_TLS=True
MAIL_USE_SSL=False
MAIL_USERNAME=peptools_med@unal.edu.co
MAIL_PASSWORD=nwis rsdj orco lnex
```

▼ **Configuración de docker compose**

docker-compose-redis.yml: redis compartido para pepgen, ampclass y pepmultifinder.


```
version: '3'

services:
  redis:
    image: redis:alpine
    container_name: redis-peptools
    networks:
      - peptools_network
    ports:
      - "6379:6379"

networks:
  peptools_network:
    external: true
```

Todos los servicios se exponen por el puerto 80 para que sean escuchados por `invepep-nginx-proxy_1` el cual es el proxy inverso que permite acceder a los servicios por medio de https

Pepmultifinder

```
version: '3'

services:
  pepmultifinder:
    build: .
    image: pepmultifinder-image
    env_file:
      - .env
    expose:
      - "80"
    volumes:
      - ./app
    environment:
      - VIRTUAL_HOST=pepmultifinder.medellin.unal.edu.co
```

- LETSENCRYPT_HOST=pepmultifinder.medellin.unal.edu.co
- LETSENCRYPT_EMAIL=peptools_med@unal.edu.co
- VIRTUAL_PORT=5001

networks:

- inverpep_default # MISMA RED que el proxy
- peptools_network

depends_on:

- redis

pepmultifinder_worker:

image: pepmultifinder-image

volumes:

- ./app

command: flask rq worker

networks:

- peptools_network

depends_on:

- redis
- pepmultifinder

redis:

image: redis:alpine

expose:

- "6379"

networks:

- peptools_network

networks:

inverpep_default:

external: true # Usa la red del proxy existente

peptools_network:

external: true

Ampclass

```
version: '3'

services:
  ampclass:
    build: .
    image: ampclass-image
    env_file:
      - .env
    expose:
      - "80"
    volumes:
      - ./app
    environment:
      - VIRTUAL_HOST=ampclass.medellin.unal.edu.co
      - LETSENCRYPT_HOST=ampclass.medellin.unal.edu.co
      - LETSENCRYPT_EMAIL=peptools_med@unal.edu.co
      - VIRTUAL_PORT=5003
    networks:
      - inverpep_default
      - peptools_network

  ampclass_worker:
    image: ampclass-image
    volumes:
      - ./app
    command: flask rq worker
    networks:
      - peptools_network
    depends_on:
      - ampclass

networks:
  inverpep_default:
    external: true
```

```
peptools_network:  
  external: true
```

Pepgen

```
version: '3'  
  
services:  
  pepgen:  
    build: .  
    image: pepgen-image  
    env_file:  
      - .env  
    expose:  
      - "80"  
    volumes:  
      - ./app  
    environment:  
      - VIRTUAL_HOST=pepgen.medellin.unal.edu.co  
      - LETSENCRYPT_HOST=pepgen.medellin.unal.edu.co  
      - LETSENCRYPT_EMAIL=peptools_med@unal.edu.co  
      - VIRTUAL_PORT=5002  
    networks:  
      - inverpep_default  
      - peptools_network  
  
  pepgen_worker:  
    image: pepgen-image  
    volumes:  
      - ./app  
    command: flask rq worker  
    networks:  
      - peptools_network  
    depends_on:  
      - pepgen
```

```
networks:
  inverpep_default:
    external: true
  peptools_network:
    external: true
```

Multipepgen API

```
version: '3.8'

services:
  multipepgen-api:
    container_name: multipepgen-api-container
    image: multipepgen-api-image
    build:
      context: .
      dockerfile: Dockerfile
    expose:
      - "80"
    environment:
      - VIRTUAL_HOST=multipepgen-api.medellin.unal.edu.co
      - LETSENCRYPT_HOST=multipepgen-api.medellin.unal.edu.co
      - LETSENCRYPT_EMAIL=peptools_med@unal.edu.co
      - VIRTUAL_PORT=9000
    networks:
      - inverpep_default # MISMA RED que el proxy
      - peptools_network
    restart: unless-stopped

networks:
  inverpep_default:
    external: true # Usa la red del proxy existente
```

```
peptools_network:  
  external: true
```

Multipepgen APP

```
version: '3.8'  
  
services:  
  multipepgen-app:  
    build:  
      context: .  
      dockerfile: Dockerfile  
    container_name: multipepgen-app-container  
    expose:  
      - "80"  
    environment:  
      - VIRTUAL_HOST=multipepgen.medellin.unal.edu.co  
      - LETSENCRYPT_HOST=multipepgen.medellin.unal.edu.co  
      - LETSENCRYPT_EMAIL=peptools_med@unal.edu.co  
      - VIRTUAL_PORT=8100  
    networks:  
      - inverpep_default # MISMA RED que el proxy  
      - peptools_network  
    restart: unless-stopped  
  
networks:  
  inverpep_default:  
    external: true # Usa la red del proxy existente  
  peptools_network:  
    external: true
```

Después de la configuración de cada docker compose se ejecutan los comandos:

```
docker compose build
docker compose up -d
```

▼ Reinicio de contenedores

1. Accede al directorio donde se encuentra el archivo `docker-compose-prod.yml` :

```
cd /ruta/del/contenedor
```

2. Detén y elimina los contenedores actuales

```
docker compose -f docker-compose-prod down
```

3. Vuelve a levantar los contenedores en segundo plano (modo *detached*):

```
docker compose -f docker-compose-prod.yml up -d
```